1. IT Gymnázium, s.r.o.

Resourcia

Maturitní práce z předmětu Informatika

Autor: Matěj Hladeček

Vedoucí práce: Ing. Daniel Kopecký

Rok: 2026

Čestné prohlášení

Prohlašuji, že jsem tuto maturitní práci vypracoval(a) samostatně a použil(a) jsem pouze uvedené zdroje.

V Praze dne 1.4.2026

Podpis: Hladeček

Anotace

Tato maturitní práce se zabývá návrhem a vývojem webové aplikace Resourcia, která funguje jako vyhledávač vzdělávacích zdrojů. Práce se věnuje analýze problémů internetem asistovaného samostudia a současných řešení. Dále obsahuje popis vybraných technologií a definuje klíčové pojmy související s projektem. Následně se zaměřuje na implementaci a návrh architektury systému. Výsledkem je funkční a perspektivní prototyp s moderním designem. Na závěr práce představuje možné směry dalšího rozvoje projektu.

Klíčová slova

webová aplikace, vzdělávací zdroje, samostudium, internetové vzdělávání, vyhledávání, filtrování, hodnocení, ASP.NET Core, Angular, PostgreSQL, autentizace

Obsah

Čestné prohlášení **1**

Anotace **2**

Klíčová slova **2**

1 Úvod **5**

2 Teoretická část **5**

2.1 Současný stav 5

2.1.1 Samostudium na internetu 6

2.1.2 Vliv internetu na vzdělávání 6

2.1.3 Tutorial hell 7

2.2 Existující řešení 8

2.2.1 Youtube 8

2.2.2 Stack Overflow 8

2.2.3 Reddit 8

2.2.4 Khan Academy 8

2.3 Požadavky 9

2.3.1 Funkční požadavky 9

2.3.2 Nefunkční požadavky 10

2.4 Použité technologie 12

2.4.1 ASP.NET a C# 12

2.4.2 Angular 12

2.4.3 Tailwind CSS 12

2.4.4 PostgreSQL 13

2.4.5 Git, Github a gitlint 13

2.4.6 Docker 14

2.4.7 Redis 14

2.5 Koncepty a architektura 14

2.5.1 Multitier architecture 14

2.5.2 Component architecture 15

2.5.3 Modulární monolit 15

2.5.4 JWT tokeny 15

2.5.5 REST 16

3 Praktická část **16**

3.1 Architektura systému 16

3.1.1 Klientská vrstva 17

3.1.2 Aplikační vrstva . . . 17
3.1.3 Datová vrstva . . . 18
3.1.4 Externí služby . . . 18
3.2 Databázový model . . . 18
3.3 Backend . . . 20
3.3.1 Struktura backendu . . . 21
3.3.2 Aplikační logika (Services) . . . 21
3.3.3 Autentizace a autorizace . . . 22
3.3.4 Cache a optimalizace výkonu . . . 22
3.3.5 Externí služby . . . 22
3.4 Frontend . . . 22
3.4.1 Struktura frontendu . . . 23
3.4.2 Routing a navigace . . . 23
3.4.3 Komunikace s backendem . . . 24
3.4.4 Stylování uživatelského rozhraní . . . 24
3.4.5 Lokalizace a internationalizace . . . 24
3.5 Use cases . . . 24
3.5.1 Nepřihlášený uživatel . . . 25
3.5.2 Přihlášený uživatel . . . 25
3.5.3 Administrátor . . . 25
3.6 Autentizace . . . 26
3.6.1 Email a heslo . . . 26
3.6.2 Google OAuth . . . 27
3.6.3 Obnova hesla . . . 27
3.7 API endpointy . . . 27
3.8 Nasazení aplikace . . . 31
3.9 Budoucí rozvoj . . . 32
3.9.1 Ukázka e-mailové komunikace . . . 33

4 Závěr **34**

5 Zdroje **36**

1 Úvod

V současné době je známo, že řadu dovedností je možné osvojit i mimo školu (Stack Overflow, 2023). Příkladem je oblast informačních technologií, kde mnoho programátorů obohatilo své schopnosti převážně samostudiem, nikoli pouze skrze vysokoškolské vzdělání.

Rozdíl mezi veřejným vzděláváním a samostudiem byl několikrát vyobrazen i v populární kultuře a odráží společenské vnímání moderního vzdělání. Současně s rozvojem internetu se počet zdrojů výrazně zvětšil a přístup k nim se zjednodušil (OECD, 2024). V online prostoru se nyní nachází velké množství užitečných webových stránek, odborné literatury i edukativních videí, které umožňují efektivní a dostupnější samostudium.

Bohužel však velké množství dostupných materiálů přináší i své problémy. Mezi nimi se vyskytuje i značný počet nekvalitních zdrojů, což může začínajícím studentům ztěžovat orientaci v možnostech. S tímto problémem se setkává při samostatném studiu řada lidí. Z tohoto důvodu byla vytvořena aplikace Resourcia.

Cílem této práce je navrhnout, naprogramovat a implementovat webovou aplikaci Resourcia, která by umožňovala vyhledávat, sdílet a hodnotit vzdělávací zdroje dostupné na internetu. Aplikace má sloužit jako nástroj pro účinnější orientaci v nepřehledné nabídce materiálů pro účely samostudia.

2 Teoretická část

Tato kapitola se zaměřuje na aktuální situaci stávajícího problému, který pomůže později definovat naše požadavky. Nejprve je popsán současný způsob, jakým lidé využívají internet pro samostudium, a jaký vliv má dostupnost online informací na proces vzdělávání. Dále diskutujeme širší kontext ze socioekonomické stránky. A nakonec předtím než definujeme požadavky se podíváme na existující řešení.

2.1 Současný stav

V dnešní době je samostudium nedílnou součástí vzdělávání v zemích s přístupem k internetu (World Bank, 2023). Tento trend přináší řadu socioekonomických příležitostí pro danou zemi, které budou diskutovány v následujících kapitolách. Z těchto výhod však mohou těžit pouze země s dostatečně rozvinutou internetovou infrastrukturou. Otevřený přístup k informacím zároveň přináší i určité nevýhody. Informace jsou navíc často prezentovány nejednotným způsobem, což může zmást začínající studenty.

2.1.1 Samostudium na internetu

Internet v posledních letech výrazně změnil způsob, jakým se lidé učí. Přestože se tyto změny zpočátku na trhu práce výrazně neprojevovaly, postupně se začínají projevovat. V některých oborech už není potřeba mít vysokoškolský titul.

Například ve Spojených státech klesl podíl pozic vyžadujících alespoň bakalářský titul z 20,4 % v roce 2019 na 17,8 % v roce 2024 (Indeed Hiring Lab, 2024). Nejde pouze o bakalářské, ale většina pozic nezmiňuje nejvyšší dosažené vzdělání. Tyto trendy naznačují přechod k hodnocení uchazečů na základě kompetencí.

V posledních letech se výrazně rozšířil přístup tzv. skills-based hiring tedy nábor založený na schopnostech, nikoliv titulech (TestGorilla, 2024). Zatímco v roce 2022 tento přístup používalo pouze 57 % zaměstnavatelů, v roce 2024 již přibližně 81 %.

Nejsou to však pouze zaměstnavatelé, kteří si uvědomují význam samostudia prostřednictvím internetu. Podle některých průzkumů přibližně 87 % britských studentů běžně používá internet k učení (Save My Exams, 2024). Dokonce 71 % se domnívá, že příprava na testy pomocí online zdrojů je účinnější než školní příprava.

Podobný trend lze pozorovat také v jednotlivých oborech. Podle StackOverflow Developer Survey 2023 pouze 50,14 % vývojářů uvádí školu jako způsob, kterým se naučili programovat (Stack Overflow, 2023). Internetové blogy, videa a fóra uvádí 80,13 % vývojářů a zajímavým údajem je také, že 51,8 % vývojářů uvádí knihy. Vývoj softwaru tak představuje obor, ve kterém je samostudium často stejně důležité nebo důležitější než tradiční vzdělávání.

Tato data od StackOverflow je však možné částečně zpochybnit, protože může docházet ke zkreslení výběru vzorku. StackOverflow je internetová platforma určená především pro programátory, kteří již aktivně využívají online zdroje při své práci nebo studiu. Je proto pravděpodobné, že respondenti tohoto průzkumu budou uvádět internetová fóra jako formu vzdělání. Přesto však tato data poskytují zajímavý pohled na význam samostudia a online zdrojů nejen v oblasti informačních technologií.

2.1.2 Vliv internetu na vzdělávání

Data z předešlé kapitoly ukazují rostoucí význam samostudia a online zdrojů při získávání nových dovedností. Je proto vhodné se podrobněji zaměřit na dopady způsobené tímto přechodem.

Internet má významný vliv nejen na způsob vzdělávání, ale i na společenský a ekonomický rozvoj (World Bank, 2023). Díky snadnému přístupu k vzdělání a informacím mohou lidé získávat nové znalosti bez formálního vzdělání a financí. Tento fenomén byl pozorován i v několika výzkumech a experimentech.

Jedním z nejdůležitějších experimentů zabývajících se samostudiem pomocí internetu byl Hole in the Wall v Indii (Mitra, 2005). V tomto experimentu byl ve veřejném prostoru připraven počítač s připojením k internetu, ke kterému měly neomezený přístup děti bez zkušeností s technologiemi. Výsledky ukázaly, že děti jsou schopny se naučit sami práci s počítačem. Zároveň navazující experimenty prokázaly jejich schopnost se naučit pokročilou biologii pomocí internetu. Děti přirozeně spolupracovaly, sdílely poznatky a společně řešily problémy,

Zároveň přístup k internetu má dopad na ekonomický rozvoj společnosti. Jedním z důležitých problémů je tzv. internetová propast (digital divide) (OECD, 2022). Jde o rozdíl mezi lidmi s přístupem k internetu a těmi, co k internetu přístup nemají. Je možné, že tento rozdíl má dopad na studijní výsledky žáků. Analýzy organizace OECD ukazují, že studenti s přístupem k internetu dosahují v průměru lepších výsledků v mezinárodních testech PISA než studenti bez tohoto přístupu (OECD, 2022). Studenti bez internetového připojení mají omezenější přístup k informacím, což může ztěžovat jejich vzdělávání.

Přístup k internetu však neovlivňuje pouze vzdělávací výsledky jednotlivců, ale také ekonomický vývoj společnosti. Rozvoj internetové infrastruktury je často spojován s hospodářským růstem. Jedním z důvodů může právě snadnější přístup ke vzdělávacím zdrojům prostřednictvím internetu. Výzkumy objevily, že frekventované užívání internetu mělo pozitivní vliv na změnu sociální třídy jednotlivců v průběhu času (Pew Research Center, 2021). Studie Světové banky navíc ukazují, že zvýšení penetrace broadband internetu o 10 procentních bodů může vést k růst HDP přibližně o 1,2 % (World Bank, 2009). Studie McKinsey Global Institute ukazuje, že internetová ekonomika tvoří přibližně 3 až 4 % HDP vyspělých států (McKinsey, 2011).

2.1.3 Tutorial hell

S rostoucím počtem vzdělávacích zdrojů na internetu se objevuje také fenomén často označovaný jako tutorial hell. Tento termín popisuje situaci, kdy se osoba dlouhodobě vystavuje učení prostřednictvím nekvalitních online tutoriálů. Tento problém je často spojován s pasivním učením (Kirschner et al., 2006), kdy replikují jednotlivé kroky prezentované v tutoriálu bez zamyšlení. Výzkumy v oblasti vzdělávání v informatice ukazují, že studenti, kteří pracují na předpřipravených příkladech, mohou mít potíže s řešení unikátních a nových problémů (Robins et al., 2003). Některé studie také upozorňují na to, že kódy jsou z tutoriálů často kopírovány bez hlubšího porozumění. To může vést k chybám a bezpečnostním problémům v reálných projektech (Acar et al., 2017). Proto je potřeba hledat kvalitnější tutoriály či zdroje vyžadující aktivnější přístup. Od toho bude sloužit Resourcia.

2.2 Existující řešení

2.2.1 Youtube

YouTube je považován za největší platformu pro sdílení videí na světě (YouTube, 2024). Platforma má přes 2,7 miliardy aktivních uživatelů měsíčně a každou minutu je na ni nahráno více než 500 hodin videoobsahu. Díky tomu představuje jeden z nejvýznamnějších zdrojů vzdělávací materiálů dostupných online (YouTube, 2024).

YouTube však není primárně zaměřen na vzdělávání, takže algoritmus je navržen tak, aby maximalizoval sledovanost a zapojení uživatelů. To znamená, že často upřednostňuje populární obsah před videi s vyšší vzdělávací hodnotou. Pro začínající studenty tak může být obtížné identifikovat skutečně kvalitní vzdělávací materiály.

2.2.2 Stack Overflow

Stack Overflow je komunitní platforma určená především pro programátory (Stack Overflow, 2023), která umožňuje uživatelům klást technické otázky a sdílet řešení problémů. Stack Overflow je součástí širší sítě webů označované jako Stack Exchange Network (Stack Overflow, 2023).

Tato síť zahrnuje řadu tematicky zaměřených stránek určených pro různé obory. Podobně jako Stack Overflow pro programátory existuje například Mathematics Stack Exchange, který je zaměřen na matematiku. Díky systému hlasování a moderace může komunita efektivně identifikovat kvalitní odpovědi. Platforma je však určena pro řešení konkrétních problémů a nikoli pro systematické učení či vyhledávání vzdělávacích zdrojů.

2.2.3 Reddit

Reddit je diskusní platforma rozdělená do tematických komunit označovaných jako subreddity (Reddit, 2024). V jednom subreddit r/selfeducation uživatelé sdílí svoje tipy na efektivní samostudium. Tato doporučení se však mohou rychle ztratit v historii, což vede k tomu, že se některé dotazy objevují opakovaně.

2.2.4 Khan Academy

Khan Academy je nezisková vzdělávací platforma poskytující bezplatné online kurzy a výuková videa (Khan Academy, 2024). Platforma nabízí strukturované studijní materiály zejména v oblasti STEM oborů. Na rozdíl od diskuzních fór nebo video platforem je obsah na Khan Academy pečlivě připravován a organizován do studijních cest. Platforma se však zaměřuje především na vlastní obsah vyrobený z vlastních financí. Nefunguje jako katalog cizích vzdělávacích zdrojů z různých částí internetu.

2.3 Požadavky

Jak vidíme z předešlé kapitoly existujících řešení, je spousta nedostatků k efektivnímu rozcestníku k vzdělávacím zdrojů. Na druhou stranu z existujících řešení můžeme čerpat spoustu inspirací k základní idei této maturitní práce.

2.3.1 Funkční požadavky

ID	Název	Popis
FR1	Registrace uživatele	Systém musí umožnit uživateli vytvořit nový uživatelský účet.
FR2	Přihlášení	Systém musí umožnit uživateli přihlásit se pomocí e-mailu a hesla.
FR3	Odhlášení	Systém musí umožnit uživateli odhlásit se ze systému.
FR4	Změna hesla	Systém musí umožnit uživateli změnit své heslo.
FR5	Obnova hesla	Systém musí umožnit uživateli obnovit zapomenuté heslo.
FR6	Profil uživatele	Systém musí umožnit uživateli zobrazit svůj profil.
FR7	Úprava profilu	Systém musí umožnit uživateli upravit informace ve svém profilu.
FR8	Smazání účtu	Systém musí umožnit uživateli smazat svůj účet.
FR9	Vyhledávání	Vyhledávání zdrojů podle názvu nebo klíčových slov.
FR10	Filtrování	Filtrování zdrojů podle kategorií a parametrů.
FR11	Detail zdroje	Zobrazení detailu vzdělávacího zdroje.
FR12	Přidání zdroje	Uživatel může přidat nový zdroj.
FR13	Úprava zdroje	Úprava existujícího zdroje autorem nebo adminem.
FR14	Smazání zdroje	Odstranění vzdělávacího zdroje.
FR15	Hodnocení	Uživatelé mohou hodnotit zdroje.
FR16	Průměrné hodnocení	Zobrazení průměrného hodnocení.
FR17	Úprava hodnocení	Uživatel může upravit nebo smazat své hodnocení.
FR18	Přiřazení kategorií	Zdroj lze přiřadit k kategoriím/filtrům.
FR19	Seznam filtrů	Zobrazení dostupných kategorií a filtrů.

ID	Název	Popis
FR20	Vícenásobné filtrování	Filtrování podle více parametrů současně.
FR21	Pořadí filtrů	Změna pořadí filtrů.
FR22	Aktivace filtru	Zapnutí/vypnutí filtru.
FR23	Smazání filtru	Odstranění filtru.
FR24	Název filtru	Změna názvu filtru.
FR25	Popis filtru	Změna popisu filtru.
FR26	Ban uživatele	Administrátor může zabanovat uživatele.
FR27	Suspendování	Administrátor může suspendovat uživatele.
FR28	Odbanování	Administrátor může odbanovat uživatele.
FR29	Zrušení suspendu	Administrátor může zrušit suspendování.
FR30	Informace o banu	Zobrazení času banu nebo suspendu.
FR31	Role admina	Povýšení uživatele na administrátora.
FR32	Odebrání admina	Odebrání administrátorských práv.
FR33	CAPTCHA	Ověření uživatele pomocí CAPTCHA.
FR34	Notifikace	Zobrazení notifikací o výsledku operací.
FR35	REST API	Backend poskytuje REST API pro práci s daty.
FR36	Soft delete	Data nejsou fyzicky smazána, pouze skryta.
FR37	Tvorba filtrů	Admin může vytvářet nové filtry.
FR38	Typ filtru	Při tvorbě se volí typ filtru.
FR39	Typy filtrů	Podpora typů: facet, string, range, boolean.
FR40	Vazba filtru	Filtr je navázán na typ určující chování.
FR41	Hodnoty filtru	Filtr umožňuje více hodnot (choices).
FR42	Výběr hodnot	Uživatel může vybrat více hodnot při filtrování.

2.3.2 Nefunkční požadavky

ID	Název	Popis
NFR1	Výkon vyhledávání	Systém musí zpracovat vyhledávací dotaz v řádu několika sekund.
NFR2	Současné požadavky	Backend musí zvládat více současných požadavků uživatelů.
NFR3	Použitelnost	Uživatelské rozhraní musí být přehledné a intuitivní.
NFR4	Kompatibilita	Aplikace musí být dostupná v moderních webových prohlížečích.

ID	Název	Popis
NFR5	Hashování hesel	Hesla musí být ukládána v hashované podobě.
NFR6	Autorizace uživatele	Přístup k funkcím pouze pro přihlášené uživatele.
NFR7	Role administrátora	Admin funkce pouze pro administrátory.
NFR8	Validace dat	Systém musí validovat vstupní data.
NFR9	Integrita dat	Zabránění uložení neplatných nebo nekompletních dat.
NFR10	Rozšiřitelnost	Architektura musí umožnit snadné rozšíření systému.
NFR11	Backend technologie	Backend v C# pomocí ASP.NET.
NFR12	Frontend technologie	Frontend pomocí Angular.
NFR13	Databáze	Použití relační databáze.
NFR14	Verzování	Zdrojový kód spravován pomocí Git.
NFR15	Branching model	Vývoj využívá větvový model.
NFR16	Feature branche	Nové funkce ve feature branchech.
NFR17	Commit pravidla	Commit zprávy kontrolovány nástrojem gitlint.
NFR18	Komponenty	Frontend využívá komponentní architekturu.
NFR19	Stylování	Použití Tailwind CSS.
NFR20	Lokalizace	Podpora více jazykových mutací.
NFR21	Oddělení textů	Texty odděleny od logiky kvůli překladům.
NFR22	Složitost operace	Změna pořadí filtrů má složitost O(1).
NFR23	Soft delete	Data zůstávají v DB, ale nejsou zobrazena.
NFR24	Docker	Aplikace musí být spustitelná pomocí Dockeru.
NFR25	VPS nasazení	Systém musí být nasaditelný na VPS.
NFR26	Produkční běh	Aplikace musí běžet v produkčním prostředí.
NFR27	Doména	Aplikace dostupná přes internetovou doménu.
NFR28	Přístupnost	Přístup přes webový prohlížeč.
NFR29	Přenositelnost	Snadné nasazení pomocí Docker kontejnerů.
NFR30	Konfigurace	Konfigurace pomocí .env souboru.
NFR31	Citlivá data	Citlivé údaje nesmí být ve zdrojovém kódu.
NFR32	Prostředí	Oddělení dev/test/prod prostředí.
NFR33	Bezpečnost .env	.env nesmí být ve veřejném repozitáři.

2.4 Použité technologie

V této kapitole budou představeny užité technologie, jejich možné náhrady a proč jsem zvolil právě tento. Budou popsány použité technologie na backend, frontend, databáze a nástroje pro správu aplikace.

2.4.1 ASP.NET a C#

Backend aplikace byl implementován v programovacím jazyce C# společně s frameworku ASP.NETu. C# je obecný vysokoúrovňový jazyk vyvíjen Microsoftem (Microsoft, 2024). Podporuje vícero paradigmat, například objektově orientované, imperativní nebo komponentově orientované programování (Microsoft, 2024). Jazyk C# se často používá pro vývoj webových aplikací nebo desktopových aplikací nebo backendových služeb v rámci platformy .NET. ASP.NET framework poskytuje nástroje pro tvorbu webových aplikací a je široce využíván (Microsoft, 2024).

Alternativou k ASP.NET by mohly být například frameworky Node.js, Django nebo Spring Boot. Tyto technologie se také běžně používají pro vývoj backendových aplikací.

ASP.NET byl zvolen především díky dobré integraci s jazykem C#, silnému typovému systému a kvalitní podpoře nástrojů pro vývoj webových aplikací. Další výhodou je vysoký výkon a kvalitní dokumentace, například prostřednictvím nástroje Swagger.

2.4.2 Angular

Frontend aplikace byl vytvořen pomocí frameworku Angular (Angular, 2024), který umožňuje vývoj dynamických webových aplikací pomocí komponentové architektury. Alternativní frameworky mohou být například, React nebo Vue.js, které jsou rovněž populární frontendové frameworky. Angular byl zvolen pro podporu TypeScriptu, Sass a svůj široký výběr vestavěných knihoven pro routing, formuláře a komunikace s backendovým API.

2.4.3 Tailwind CSS

Pro stylování uživatelského rozhraní existuje několik přístupů a nástrojů. Patří mezi ně čisté CSS, CSS preprocesory nebo CSS frameworky. Používají se často nástroje jako Sass, Bootstrap, Material UI nebo Tailwind CSS (Tailwind CSS, 2024).

Použití čistého CSS je nejflexibilnější, jelikož vývojář má plnou kontrolu nad definicí všech stylů. Bohužel toto znamená, že CSS soubory mohou být rozsáhlé a vyžaduje pečlivou organizaci souborů.

Jedním ze způsobů, jak zlepšit organizaci CSS kódu, je použití CSS preprocesorů, například Sass nebo Scss (Sass, 2024). Sass rozšiřuje možnosti klasického CSS a přidává proměnné,

mixiny nebo vnořené selektory. Zároveň nabízí stejnou nebo lepší flexibilitu. Poté lze psát přehlednější stylový kód, který je následně kompilován do CSS.

Přehlednější kód se dá zajistit i metodikou pojmenování tříd, například BEM (Block Element Modifier) (BEM, 2024). BEM definuje konvenci pro pojmenování tříd a pomáhá tak rozlišit části uživatelského rozhraní.

Další možností jsou CSS komponentní frameworky, které poskytují předpřipravené komponenty uživatelského rozhraní (Bootstrap, 2024). Mezi tyto frameworky patří například Bootstrap nebo Material UI. Ty poskytují předpřipravené komponenty uživatelského rozhraní. Rychle se nasazují poté tlačítka, textové pole až po carousely a dashboard. Vývoj frontend aplikací se s těmito frameworky může výrazně urychlit. Nevýhodou je omezená flexibilita a také skutečnost, že aplikace pomocí těchto frameworků mají často velmi podobný design.

Tailwind CSS je moderní utility-first framework (Tailwind CSS, 2024), který poskytuje komponenty, ale dobře se hodí pro vývoj aplikací s komponentovou architekturou. Například vhodnou kombinací je Angular a Tailwind. Místo předpřipravených komponent poskytuje velké množství malých utility tříd. Umožňují tak upravovat základní stylové atributy přímo v HTML šablonách. Tento přístup umožňuje rychle vytvářet vlastní design bez nutnosti psaní velkého množství CSS. Výhodou je také, že Tailwind vytváří co nejmenší CSS soubor. Ve některých případech může být dokonce menší než u projektu s čistým CSS. Nevýhodou může být nepřehledné HTML kódy, které se dají zpřehlednit s komponentovou architekturou.

2.4.4 PostgreSQL

PostgreSQL je neplacená a opensource relační databáze (PostgreSQL, 2024). Podporuje tvorbu vlastních objektů, děditelnost, polymorphismus, ACID transakce a další funkce. Alternativou k PostgreSQL by mohly být například MySQL, MariaDB nebo některé NoSQL databáze, například MongoDB. PostgreSQL byla zvolena díky své stabilitě a jednoduché integraci s ASP.NET.

2.4.5 Git, Github a gitlint

Pro správu a verzování zdrojového kódu byl použit Git. Git je open-source systém pro správu verzí (Git, 2024), který umožňuje sledovat změny v souborech během vývoje softwaru. Umožňuje vývojářům pracovat na projektech paralelně, spravovat historii změn a případně vrátit změny.

GitHub je online platforma pro hostování Git repozitářů a spolupráci na vývoji softwaru (GitHub, 2024). GitHub byl využit jako centrální repozitář zdrojového kódu pro Resourcii.

Nástroj gitlint byl použit pro kontrolu formátu commit zpráv. Gitlint automaticky ověřuje, zda commit zprávy splňují přednastavený vzor. Tím je zajištěna lepší orientace v historii změn.

2.4.6 Docker

Docker je open-source aplikace určená ke kontejnerizaci aplikací (Docker, 2024). Umožňuje aplikaci zabalit se všemi potřebnými závislostmi. V projektu Resourcia je Docker je využíván pro snadné nasazení aplikace na server. Možnou alternativou je například Kubernetes.

2.4.7 Redis

Redis je open-source cachovací databáze (Redis, 2024), která slouží jako rychlejší uložiště dat oproti databázi. Data jsou ukládána v operační paměti, což umožňuje velmi rychlý přístup k nim. Redis se často používá na zrychlení poměrně pomalých nebo zatížených query.

2.5 Koncepty a architektura

2.5.1 Multitier architecture

Multitier architecture, také označovaná jako n-tier architecture, je architektonický přístup k vývoji aplikací. Při něm je aplikace rozdělena do více vrstev, které spolupracují (Fowler, 2002). Každá vrstva má odlišné odpovědnosti a liší se přístupností. Toto rozdělení zlepšuje přehlednost, udržovatelnost a škálování aplikace.

Typicky se aplikace skládá z tří hlavních vrstev. Jde o prezenční vrstvu (frontend), aplikační vrstvu obsahující business logiku a datovou vrstvu zajišťující práci s databází (Fowler, 2002).

V projektu Resourcia je tento princip využit tak, že frontend komunikuje s backendovým API zvaným Resourcia.Api. Backend obsahuje logiku aplikace a komunikuje s datovou vrstvou implementovanou v Resourcia.Data a PostgreSQL.

Client-server architecture Client-server architektura je model posílání zpráv v distribuované aplikaci, který rozděluje systém na dvě hlavní role: klienta a server (Tanenbaum, 2016). Klient je aplikace, která odesílá požadavky serveru a prezentuje výsledky uživateli. Server tyto požadavky přijímá a odpovídá na ně.

Komunikace mezi klientem a serverem většinou probíhá na bázi síťových protokolů, jako například HTTP, HTTPS nebo WebSocket (Fielding, 2000). Tento model dovoluje rozdělit uživatelské rozhraní od aplikační logiky, což zajišťuje určitou bezpečnost aplikace.

V projektu Resourcia je webová aplikace klientem, který komunikuje s backendovým Resourcia.Api. Backend zpracovává požadavky, může pracovat s daty a následně vrací klientovi odpověď.

2.5.2 Component architecture

Komponentová architektura je přístup jak navrhovat aplikace, při kterém je systém rozdělen na menší samostatné části - komponenty (Szyperski, 2002) (Gamma et al., 1994). Každá komponenta má vlastní povinnosti, například v rámci UI může jít o jednotlivé části uživatelského rozhraní. Tento přístup zvyšuje přehlednost zdrojového kódu, umožňuje jednotlivé části využívat znovu na vícero místech a dodržuje tak princip DRY - don't repeat yourself.

Angular framework je na tomto principu založen. Umožňuje vám tvořit komponenty a následně je importovat do jednotlivých stránek. Každá komponenta obsahuje šablonu (HTML), logiku (TypeScript) a stylování (CSS a SCSS). Komponenty následně spolu komunikují pomocí vstupů a výstupů.

Frontendová část Resourcia byla rozdělena do několika komponent. Mezi tyto komponenty patří například tlačítko, textový vstup a hero banner.

2.5.3 Modulární monolit

Modulární monolit je architektonický přístup, při kterém je aplikace vyvíjená jako jeden celek, ale její vnitřek je rozdělen do samostatných modulů (Fowler, 2015). Každý modul obsahuje část funkcionality celé aplikace a komunikuje s ostatními moduly s definovaným rozhraním.

Na rozdíl od klasického monolitu, kde je veškerá logika v jediné části kódu (Fowler, 2015), modulární monolit rozděluje aplikaci do menších modulů s vlastním programem.

Na rozdíl od microservices architektury běží všechny moduly v rámci jedné aplikace. Tento přístup umožňuje využívat jednoduchosti monolitické aplikace, ale zároveň umožňuje lepší organizaci kódu než v microservices.

V projektu Resourcia je backend rozdělen do modulů jako jsou například: Controllers, Services a Data layer. Tyto moduly rozdělují aplikaci na jednotlivé části logiky a zlepšují přehlednost projektu.

2.5.4 JWT tokeny

JWT (JSON Web Token) je způsob bezpečného předávání a uchování informací posílaných mezi klientem a serverem ve formátu JSON (Jones et al., 2015). Token může obsahovat

různé informace, nejčastěji identitu uživatele nebo jeho oprávnění.

Pro zajištění integrity dat je token podepsán serverem pomocí kryptografického podpisu. Tento mechanismus pomáhá předcházet útokům, jako je neoprávněná manipulace s tokenem (tzv. token tampering) nebo vydávání se za jiného uživatele (impersonation) (OWASP, 2023). Díky tomu server dokáže autorizovat uživatele a případně zjistit zda byl token změněn. Pokud by došlo k jakékoliv záměně informací, podpis by nebyl platný a server by token zamítnul.

JWT tokeny jsou často implementovány v moderních webových aplikacích.

2.5.5 REST

REST API (Representational State Transfer Application Programming Interface) je architektonický styl komunikace mezi klientem a serverem pomocí HTTP (Fielding, 2000). API poskytuje endpointy, na kterých klient má možnost získávat nebo měnit data.

- **GET**: Získání dat
- **POST**: Vytvoření nového záznamu dat
- **PUT/PATCH**: Úprava existujícího záznamu
- **DELETE**: Žádost o vymazání záznamu

Lze si zároveň všimnout srovnalosti s CRUD, který definuje čtyři základní práce s daty: CREATE, READ, UPDATE, DELETE.

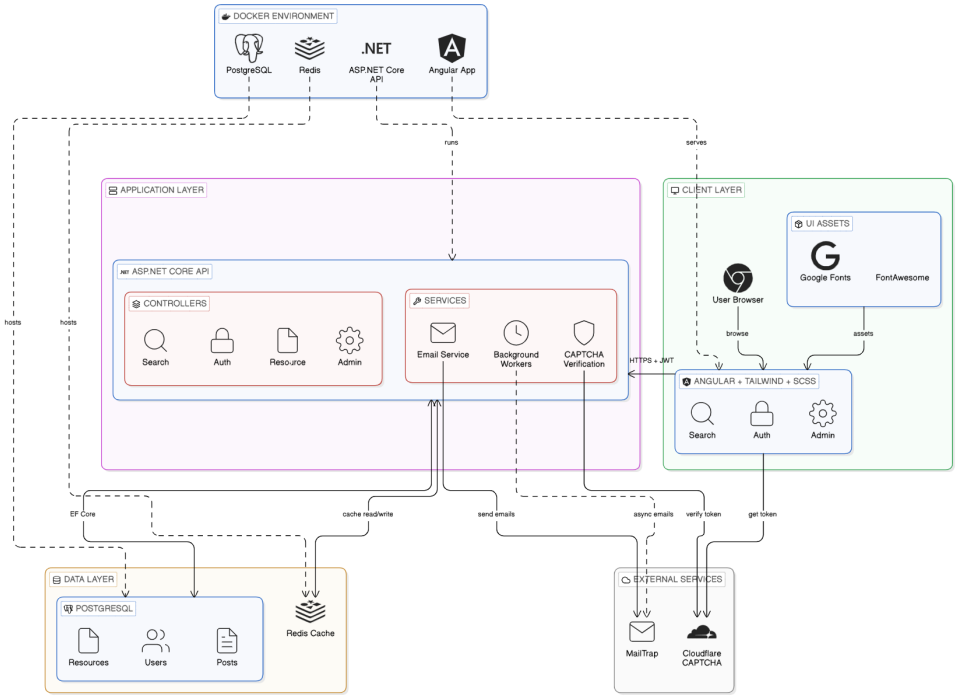
V projektu Resourcia backend poskytuje REST API. API slouží jako prostředník mezi frontendovou aplikací a databází.

3 Praktická část

3.1 Architektura systému

Architektura je navržena tak, aby byla rozdělena do více vrstev (Fowler, 2002). Systém se tedy skládá z klientské vrstvy, aplikační vrstvy, datové vrstvy a externích služeb. Jednotlivé části spolupracují pomocí předem definovaných rozhraní, což zabraňuje nepřehlednosti a udržuje systém bezpečný.

Na obrázku 1 je znázorněn stávající návrh architektury. Frontendová část je implementovaná pomocí Angularu a funguje jako klientská vrstva. S tou může uživatel komunikovat s pomocí prohlížeče jako například Google Chrome. Backendová část je implementovaná v C# s ASP.NET CORE a poskytuje API. Data následně ukládá v databázi PostgreSQL.



Obrázek 1: Ukázka rozhraní aplikace

Pro urychlení některých query je použitá cache. V tomto případě zajišťuje cache vrstvu Redis. Systém dále používá externí služby jako Cloudflare CAPTCHA nebo MailTrap.

3.1.1 Klientská vrstva

Klientská vrstva je tvořena frontendem vytvořeným s pomocí frameworku Angular. Umožňuje vizualizaci dat klientovi a ovládání s uživatelským rozhraním. Toto rozhraní je stylizováno hlavně pomocí Tailwind CSS, v některých vzácných případech také pomocí SCSS. Ikonografie je čerpaná z knihovny Font Awesome (Font Awesome, 2024) a fonty ze služby Google Fonts (Google Fonts, 2024).

Frontend zajišťuje většinu klíčových funkcí aplikace, například vyhledávání zdrojů, autentizace a administrátorské rozhraní. Následně komunikuje s backendem skrze HTTPS pomocí REST API (Fielding, 2000).

3.1.2 Aplikační vrstva

Aplikační vrstva je sestavena pomocí ASP.NET CORE API. Zpracovává požadavky od klienta, obsahuje logiku systému a zajišťuje práci s databází, cache vrstvou i externími službami.

Backend je rozdělen na části kódu zvané kontrolery a služby. Kontrolery jsou vstupními body API a zajišťují příjem a směrování HTTP požadavků. Jednotlivé kontrolery jsou rozdělené na základě zaměření. Můžeme tak mít kontroler zaměřený například na autentizaci,

vyhledávání, správu zdrojů nebo administraci. Servisní vrstva je naopak složena ze služeb a obsahuje veškerou logiku aplikace, například práci s e-maily, práce s cache a ověřování CAPTCHA tokenů.

3.1.3 Datová vrstva

Datová vrstva je tvořena databází PostgreSQL a cache vrstvou Redis. PostgreSQL zastupuje roli jako úložiště dat aplikace. Jsou v ní uchování uživatelé, zdroje, recenze, filtry a další aplikační data. Přístup k databázi je možný pouze prostřednictvím backendu. Zajišťuje tak celkovou bezpečnost systému.

Redis je využíván jako cache vrstva pro urychlení odpovědí na časté požadavky (Redis, 2024). V tomto projektu je využíván zejména při vyhledávání nebo načítání nejčastěji navštěvovaných zdrojů. Před zpracováním požadavku v databázi se nejprve ověří zda-li nejsou data dostupná v cache. Pokud nejsou, načte je z databáze a uloží je do Redis pro budoucí použití. Tento přístup přispívá k rychlejším odpovědím a snížení zátěže na databázi (Redis, 2024).

3.1.4 Externí služby

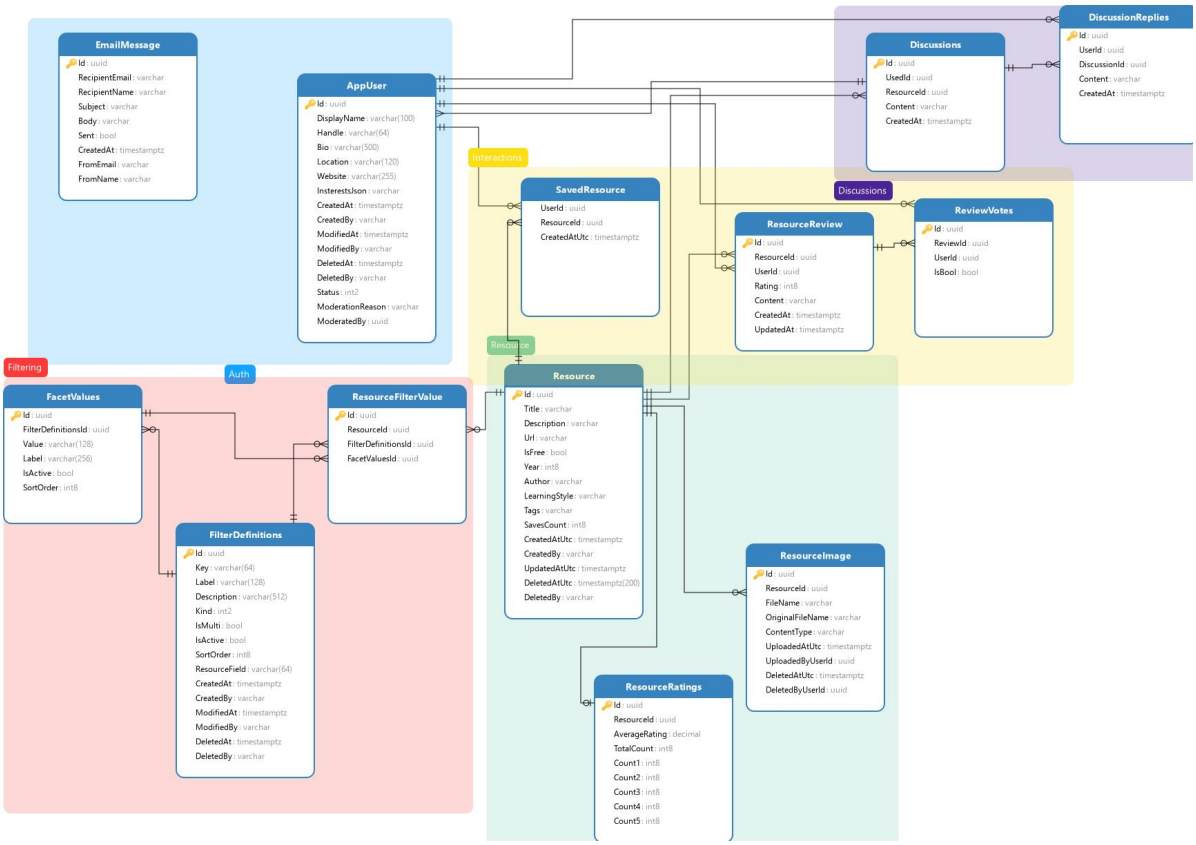
Součástí architektury jsou také externí služby. Rozšiřují funkcionalitu aplikace. Pro ochranu přihlašování je využita služba Cloudflare CAPTCHA (Cloudflare, 2024). Frontend nejprve obdrží token od Cloudflare, který následně odešle backendové vrstvě. Backend následně ověří token u Cloudflare. Pokud Cloudflare úspěšně ověří token backend dokončí proces autentizace.

Systém dále využívá Google OAuth (Google, 2024), což umožňuje přihlášení uživatelů pomocí třetí strany. Frontend umožní uživateli zahájit proces přihlášení přes třetí stranu a následně dojde k ověření identity prostřednictvím Google. Nakonec backend na základě obdržených informací vytvoří nebo dohledá účet jim odpovídající. Poté je uživateli dán autentizační token.

Pro testování e-mailové komunikace byla použita služba MailTrap (Mailtrap, 2024). Aplikace tuto službu používá například při odesílání e-mailů týkajících se přihlášení.

3.2 Databázový model

Databázový model aplikace Resourcia je implementován v databázi PostgreSQL. Jeho cílem je umožnit přehledné ukládání dat o uživatelích, vzdělávacích zdrojích, hodnoceních, filtrech a dalších entitách systému. Veškeré entity se nacházejí ve složce Entities v Resourcia.Data. Jejich návrh a obecně i celkový návrh databáze vychází z funkčních požadavků (Elmasri & Navathe, 2016).



Obrázek 2: Databázový model aplikace Resourcia

Klíčovou entitou aplikace je uživatel, neboli User. Uživatel má vlastní účet, do kterého se může přihlašovat, upravovat svůj profil a vykonávat činnosti vyžadující autorizaci. Ne všechny funkce webové aplikace jsou dostupné bez přihlášení, což je dále popsáno v kapitole Use Cases. Dále obsahuje atributy potřebné pro vytvoření profilové stránky či umožňuje administrátorům omezovat jejich práva.

Další klíčovou entitou jsou samotné učební zdroje, tedy Resource. Tato entita obsahuje základní informace o zdroji, zatímco méně často využívané údaje mohou být obsaženy v dalších navázaných entitách. Samotná entita Resource nese například informace o doméně, názvu zdroje, datu založení nebo jeho zakladateli.

Souhrnné informace o hodnocení zdroje jsou v zcela jiné entitě ResourceRatingSummary. Jejím hlavním účelem je poskytnout shrnutí hodnocení daného zdroje (Elmasri & Navathe, 2016). Neobsahuje tedy jednotlivé recenze ani konkrétní hodnocení; lze z ní vyčíst například jejich průměr, celkový počet nebo zastoupení od každé úrovně hodnocení. Tato entita se aktualizuje pokaždé, když je přidáno nebo odstraněno hodnocení.

Pro ukládání jednotlivých hodnocení je určena entita ResourceReview. Ta obsahuje identifikaci uživatele, který recenzi napsal, tak její hodnocení i textový obsah. Jednotliví uživatelé mohou zároveň hodnotit užitečnost recenze pomocí ReviewsVotes.

Pro ukládání obrázků a vizuálních náhledů pro jednotlivé zdroje slouží entita ResourceImages. Tato entita umožňuje přidat ke konkrétnímu zdroji jeden nebo vícero obrázků, například screenshoty. Každý záznam obsahuje odkaz na uložený soubor, případně další metadata, jako pořadí zobrazení nebo informace o smazání. Avšak toto nebylo zatím implementováno na frontendu.

Aby systém splňoval funkční požadavek ukládání oblíbených zdrojů, vznikla entita SavedResources. Jde o spojovací tabulky mezi Users a Resources (Elmasri & Navathe, 2016). Umožňuje tak uživatelům si vytvořit vlastní seznam zdrojů. Záznamy unikátních dvojic zároveň obsahují informaci o času uložení.

Nakonec systém umožňuje vést pod zdrojem diskuzi pomocí entity ResourceDiscussion. Její účel je podpořit komunikaci uživatelů nad daným zdrojem, například při dotazech na podrobnosti. Jelikož diskuse obvykle obsahuje odpovědi, tento je tento model rozšířen o entitu DiscussionReplies. Ta představuje jednotlivé odpovědi v diskuzním vlákně.

Protože bylo cílem měnit filtrační systém dynamicky z administrátorského rozhraní, byla potřeba vytvořit pro filtry entitu zvanou FilterDefinitions. Tato entita pomáhá definovat název a typ filtru. Možné typy jsou například:

- **String**: textová hodnota, která je pro každý zdroj unikátní, například autor nebo doména
- **Range**: celé číslo, které se pohybuje v nějakém rozmezí, například rok založení
- **Bool**: logická hodnota, která pomáhá určit skutečnost ohledně daného zdroje, například zda zdroj využívá AI, je open-source nebo funguje offline
- **Facet**: výběrový filtr s předem definovanými možnostmi, tzv. faceted search, který umožňuje filtrování podle více atributů (Tunkelang, 2009), například oblast zaměření nebo jazyk

Pro reprezentaci jednotlivých možných hodnot facet filtrů slouží entita FacetValues. Ta obsahuje konkrétní hodnoty, kterých může daný filtr může nabývat. Propojení všech filtrů, které nemají vlastní atribut v Resource, je realizované pomocí entity ResourceFilterValues. Tato struktura umožňuje jak přiřazení vícero facet hodnot stejného filtru ke stejnému zdroji. Zároveň tento systém zachovává flexibilitu i přes veškeré změny či rozšiřování.

3.3 Backend

Backend aplikace je vyvíjen pomocí technologie ASP.NET Core v jazyce C#. Funguje jako aplikační vrstva systému, která zpracovává požadavky přicházející od klienta. Zároveň obsahuje logiku aplikace a zajišťuje komunikaci s databází i externími službami.

3.3.1 Struktura backendu

Jelikož je backend navržen jako modulární monolit (Fowler, 2015), je tato vrstva rozdělena do několika částí:

- **Controllers** – vstupní brány aplikace, které zpracovávají HTTP požadavky
- **Services** – obsahují aplikační logiku
- **Models** – datové přenosné modely (třídy s definovanými fieldy)
- **Options** – třídy pro práci s externími službami
- **Utils** – pomocné třídy pro opakovanou logiku
- **Data layer (Resourcia.Data)** – práce s databází a definice entit

Kontrolery byly rozděleny na základě funkčních oblastí systému:

- **AuthController** – autentizace uživatelů
- **ResourceController** – práce se vzdělávacími zdroji
- **SearchController** – vyhledávání
- **ReviewsController** – správa hodnocení
- **AdminUsersController, AdminFilterController** – administrátorské operace

Takové rozdělení pomáhá s udržitelností a přehledností kódu (Fowler, 2015).

3.3.2 Aplikační logika (Services)

Veškerá business logika je obsažena ve službách:

- **ProfileService** – správa uživatelských profilů
- **ReviewService** – práce s hodnocením zdrojů
- **CacheService** – komunikace s Redis cache
- **CaptchaService** – ověření Cloudflare CAPTCHA
- **EmailSenderService** – odesílání e-mailů
- **OAuthService** – přihlášení pomocí třetí strany

Kontrolery pouze přijímají požadavky a delegují jejich zpracování na služby.

3.3.3 Autentizace a autorizace

Pro ověřování uživatelů jsou použity JWT (JSON Web Tokeny) (Jones et al., 2015). Po přihlášení server vytvoří token obsahující uživatelské informace. Tento token je následně zasílán při každém požadavku klienta, backend ověřuje jeho platnost a rozhoduje o přístupu k chráněným endpointům.

Systém podporuje:

- registraci a přihlášení pomocí e-mailu a hesla
- obnovu zapomenutého hesla
- přihlášení přes externí službu (Google OAuth)

Pro zvýšení bezpečnosti je využívána také CAPTCHA (Cloudflare, 2024).

3.3.4 Cache a optimalizace výkonu

Pro urychlení odpovědí serveru je implementována cache pomocí technologie Redis (Redis, 2024). Cache se využívá zejména při:

- vyhledávání zdrojů
- načítání často navštěvovaných zdrojů

Před každým dotazem do databáze backend nejprve kontroluje cache. Pokud jsou data dostupná, vrací je přímo z ní, čímž se snižuje zatížení databáze a zrychlují odpovědi (Redis, 2024).

3.3.5 Externí služby

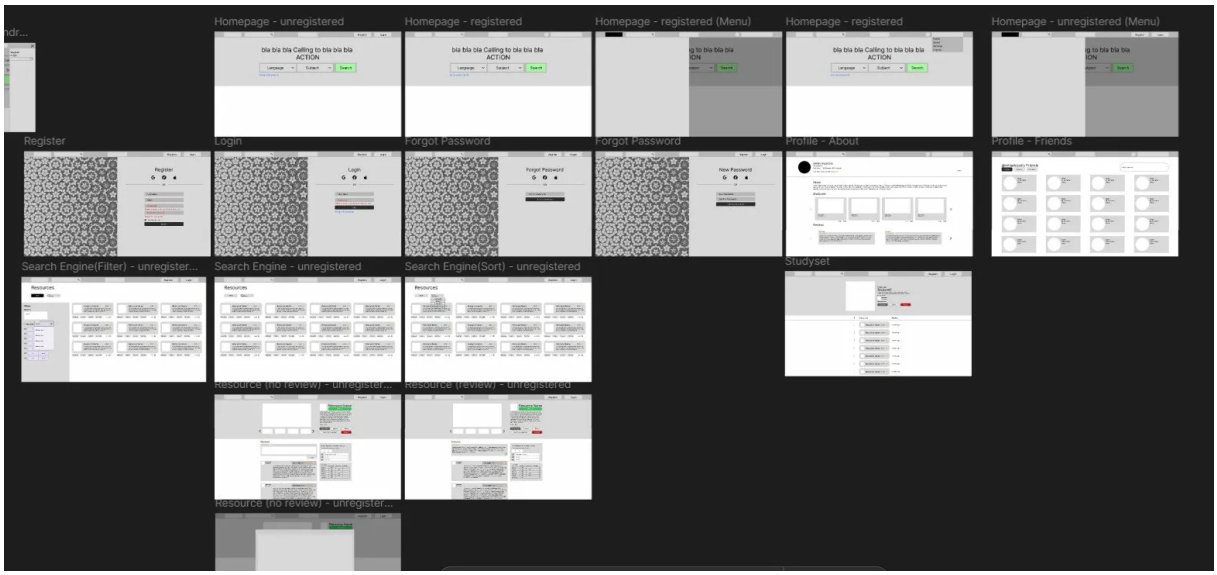
Backend komunikuje s následujícími externími službami:

- **Cloudflare CAPTCHA** – ochrana proti automatizovaným útokům
- **Google OAuth** – přihlášení pomocí externí služby (Google, 2024)
- **MailTrap** – odesílání e-mailů

Konfigurace těchto služeb je řešena pomocí tříd `Options` a proměnných `secrets`, což umožňuje snadné nasazení bez nutnosti úprav zdrojového kódu.

3.4 Frontend

Frontend je implementován pomocí frameworku Angular (Angular, 2024) a představuje prezentační vrstvu systému. Uživatel může využívat webovou aplikaci nebo rozšíření



Obrázek 3: Wireframy uživatelského rozhraní aplikace Resourcia

prohlížeče Chrome. Rozšíření prohlížeče má omezenější funkcionalitu než webová aplikace. Hlavním úkolem webové aplikace je zobrazení dat a komunikaci s backendovým REST API.

Při tvorbě frontendu byla prioritou přehlednost UI, užitečnost komponent a rozdělení logiky. Frontend tak slouží jako prostředník pro komunikaci s backendem.

3.4.1 Struktura frontendu

Frontend je navržen na základě komponentové architektury (Szyperski, 2002), na které byl Angular přizpůsoben. Aplikace byla rozdělena na menší kusy kódu podobně jako byl backend. Ty představují jednotlivé části UI. Každá komponenta má šablonu, logiku a stylování (HTML, TS a SCSS).

Tento přístup zjednodušuje údržbu frontendu podobně jako backendu. Umožňuje opakované použití již vytvořených částí. Typické komponenty jsou například navigační prvky, tlačítka, karty zdrojů nebo filtrační panel. Frontend podobně jako backend obsahuje služby, které umožňují komunikaci s backendovým API.

3.4.2 Routing a navigace

Navigace mezi jednotlivými stránkami uživatelského rozhraní je řízena pomocí Angularu Routeru. Ten umožňuje přecházet mezi stránkami bez znovunačítání celé aplikace. Uživatel tak může s lehkostí přejít například z domovské stránky na vyhledávací stránku.

Výhodou routingu je především komfort uživatele, který je způsoben rychlejší načítáním stránek. Routing zároveň umožňuje omezit přístupy na stránky na základě práv uživatele.

3.4.3 Komunikace s backendem

frontend komunikuje s backendem prostřednictvím vestavěného nástroje HttpClient. Pomocí něj frontend získává data z databáze o zdrojích, profilech, recenzích, filtrech a dalších entitách aplikace.

Podobným způsobem frontend může odesílat data na backend. To dělá například při autorizaci uživatele, vyhledávání zdrojů nebo při administrátorské činnosti. Veškerá komunikace probíhá přes HTTPS, což zajišťuje bezpečnost komunikace (Rescorla, 2018) a zabraňuje útokům typu Man-in-the-middle.

3.4.4 Stylování uživatelského rozhraní

Pro stylování webové aplikace byl využit převážně Tailwind. Tento přístup umožňuje rychle vytvářet uživatelské rozhraní. Výhodou je rychlejší vývoj, méně vlastního CSS a dobrá spolupráce s komponenty.

V některých případech byla potřeba doplnit stylování o SCSS. Pro vizuální stránku aplikace byly dále využity knihovny jako Font Awesome pro ikony. Pro písma byl zprostředkován Google Fonts.

3.4.5 Lokalizace a internationalizace

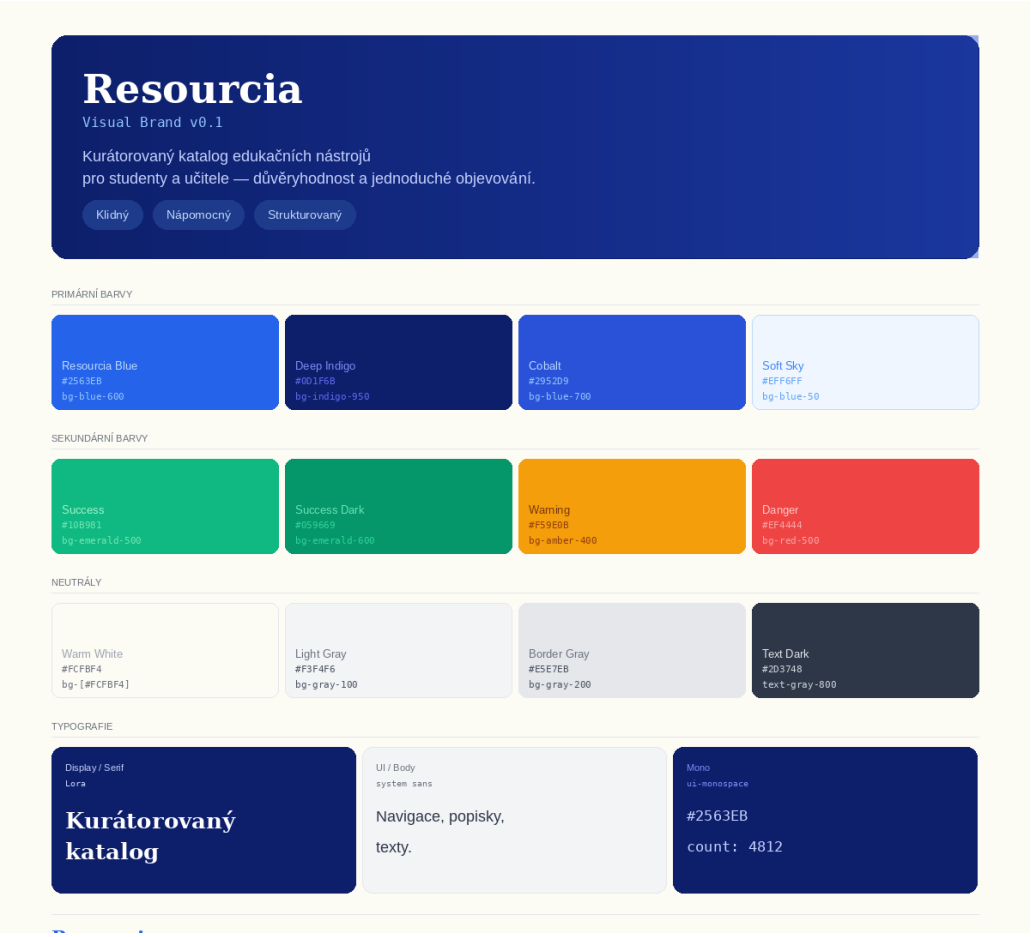
Resourcia je navržena, aby v budoucnu podporovala více jazyků. Ideálně, aby byly splněny požadavky NFR20 (Lokalizace) a NFR21 (Oddělení textů), musí být striktně oddělené překlady od zdrojového kódu.

Pro implementaci lokalizace ve frontendové části byla využita knihovna i18n. Umožňuje vytvořit zdrojový soubor pro překlad (zpravidla ve formátu .xlf) z HTML tagů obsahujících atribut "i18n". Vytvořením kopií můžeme poté lokalizovat daný jazyk.

Tato funkce byla implementována v rané fázi projektu a později postupné rozšiřování aplikace vyvolalo potřebu. To vedlo k problému, že vygenerování nového souboru s knihovnou i18n by však znamenalo riziko ztráty již hotových překladů. Proto ke knihovně i18n byla integrována ng-extract-i18n-merge (ngi18nmerge, 2024), která spojuje staré překlady s novými.

3.5 Use cases

Projekt Resourcia rozlišuje tři typy uživatelů. Každá další role má zpravidla více pravomocí než ta předchozí.



Obrázek 4: Vizuální identita aplikace Resourcia — barevná paleta a typografie

3.5.1 Nepřihlášený uživatel

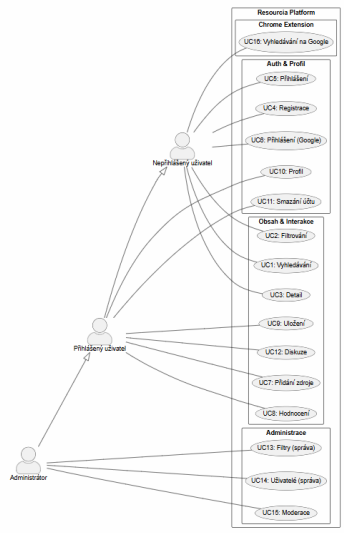
Nepřihlášený uživatel smí procházet a vyhledávat zdroje. Může si zobrazit detailní stránku zdroje a profily přihlášených uživatelů. Filtrování zdrojů je mu rovněž zpřístupněno. Nemá však žádné oprávnění k zápisu. Výjimkou jsou procesy autorizace uživatele.

3.5.2 Přihlášený uživatel

Přihlášený uživatel má všechny pravomoci nepřihlášeného uživatele, s výjimkou přístupu k některým autorizačním formulářům (např. registrace již přihlášeného uživatele). Navíc může do systému přidávat nové zdroje, hodnotit je a ukládat si je.

3.5.3 Administrátor

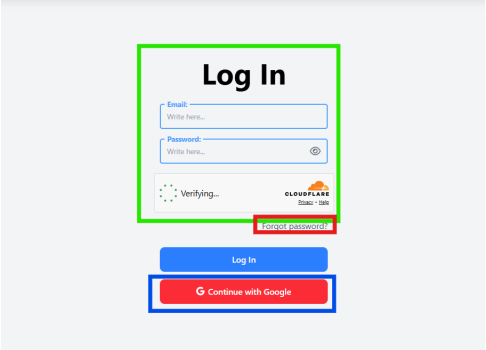
Administrátoři mají ze všech uživatelů nejvyšší pravomoci. Mohou provádět stejné akce jako přihlášení uživatelé, ale navíc mají práva potřebná pro udržení pořádku na stránce. Mezi ně patří banování uživatelů, mazání zdrojů či přidávání nových filtrů.



Obrázek 5: Diagram případů užití Resourcie - UML

3.6 Autentizace

Jelikož se pravomoci liší na základě toho, zda je uživatel přihlášený, musí systém obsahovat spolehlivé autentizační mechanismy. V této kapitole jsou představeny tři základní funkce aktuálního autentizačního systému.



Obrázek 6: Ukázká autorizace z prezentační vrstvy

3.6.1 Email a heslo

Nejjednodušší, nejstarší a možná i archaický způsob přihlašování byl implementován jako první. Jedná se o klasické přihlášení pomocí emailu a hesla. Viz obrázek 6., jedná se o postup v zeleném rámečku. Standardní přihlašovací proces probíhá v následujících krocích:

Uživatel vyplní přihlašovací formulář a projde CAPTCHA ověřením. Frontend následně odešle údaje společně s CAPTCHA tokenem na backend. Backend nejprve ověří platnost tokenu a poté porovná zadané heslo se zahashovaným a osoleným heslem uloženým v databázi. V případě, že obě kontroly proběhnou úspěšně, server vygeneruje JWT token obsahující identitu uživatele a jeho roli. Tento token je odeslán na frontend, kde je uložen

do lokálního úložiště prohlížeče. Token je následně přikládán ke každému požadavku v hlavičce Authorization.

JWT tokeny mají omezenou platnost 14 dní. Po jejím vypršení je uživatel vyzván k opětovnému přihlášení. Backend kontroluje jak integritu tokenu, tak jeho platnost. Pokud je token pozměněn nebo vypršel, je požadavek zamítnut s chybou 401 Unauthorized.

Hesla jsou v databázi ukládána v hashované podobě. Při vytvoření účtu je heslo nejprve zahashováno a následně osoleno. Teprve poté je bezpečně uloženo do databáze. Důvodem je minimalizace dopadů případného úniku dat. Proto by měli vývojáři dbát na správnou implementaci těchto postupů, případně využít ověřená řešení třetích stran.

3.6.2 Google OAuth

Systém podporuje také přihlášení pomocí OAuth, konkrétně Google OAuth 2.0. Uživatel klikne na tlačítko „Login with Google“ a je přesměrován na autorizační stránku Googlu. Po úspěšném ověření Google vrátí autorizační kód, který je následně zpracován backendem. Backend z tohoto kódu získá základní informace o uživateli, jako je email nebo jméno. Na základě těchto údajů dokončí autentizaci a vydá JWT token. Viz obrázek 6., jedná se o postup v modrém rámečku.

3.6.3 Obnova hesla

V případě zapomenutého hesla má uživatel možnost jeho obnovení. Po zadání emailové adresy obdrží odkaz pro resetování hesla. Tento odkaz obsahuje bezpečnostní token s časově omezenou platností, což zabraňuje jeho zneužití po delší době. Viz obrázek 6., jedná se o postup v červeném rámečku.

3.7 API endpointy

Tabulka 3: Přehled API endpointů aplikace Resourcia

Endpoint	**Auth**	**Popis**
Admin Filters		
GET /api/admin/filters	Admin	List all active filter definitions
POST /api/admin/filters	Admin	Create a new filter definition
PUT /api/admin/filters/{id}	Admin	Update an existing filter definition

Pokračování na další straně

Tabulka 3: Přehled API endpointů aplikace Resourcia

Endpoint	**Auth**	**Popis**
PATCH /api/admin/filters/{id}	Admin	Patch an existing filter definition
PATCH /api/admin/filters/{id}/toggleActivity	Admin	Toggle filter active status flag
PATCH /api/admin/filters/reorder	Admin	Reorder filters using fractional indexing
DELETE /api/admin/filters/{id}	Admin	Soft-delete a filter definition record
Admin Resources		
GET /api/admin/resources	Admin	List resources with admin metadata
DELETE /api/admin/resources/{id}	Admin	Soft-delete a resource entry record
Admin Users		
GET /api/admin/users	Admin	List users with activity summaries
POST /api/admin/users/{id}/suspend	Admin	Suspend a user account temporarily
POST /api/admin/users/{id}/unsuspend	Admin	Remove a user suspension status
POST /api/admin/users/{id}/ban	Admin	Ban a user account permanently
POST /api/admin/users/{id}/unban	Admin	Remove a user ban status
DELETE /api/admin/users/{id}	Admin	Deactivate a user account record
Auth		
POST /api/Auth/Register	No	Register user and send confirmation
POST /api/Auth/ResendEmail	No	Resend confirmation email if pending

Pokračování na další straně

Tabulka 3: Přehled API endpointů aplikace Resourcia

Endpoint	**Auth**	**Popis**
POST /api/Auth/ValidateToken	No	Confirm email token and issue JWT
POST /api/Auth/Login	No	Authenticate user and issue JWT
POST /api/Auth/ForgotPassword	No	Send password reset email link
POST /api/Auth/ResetPassword	No	Reset password using provided token
POST /api/Auth/RefreshToken	No	Rotate refresh token and JWT
GET /api/Auth/UserInfo	Yes	Return current authenticated user info
POST /api/Auth/Logout	Yes	Revoke current refresh token cookie
GET /api/Auth/TestMeBeforeLoginAndAfter	Yes	Test protected auth endpoint reachability
GET /api/Auth/TestMail	No	Queue a test email message
Filters		
GET /api/Filter	No	List active filters with values
GET /api/Filter/{filterName}	No	List values for a filter
Posts		
POST /api/Post	Yes	Create a new post entry
PUT /api/Post/{id}	Yes	Update an existing post entry
GET /api/Post/{id}	No	Get a single post by id
GET /api/Post	No	List all posts with authors
DELETE /api/Post/{id}	No	Delete a post entry by id
Profile		
GET /api/Profile/{identifier}	No	Get public profile by handle
PATCH /api/Profile/me	Yes	Update current user's profile details

Pokračování na další straně

Tabulka 3: Přehled API endpointů aplikace Resourcia

Endpoint	**Auth**	**Popis**
DELETE /api/Profile/me	Yes	Delete current user's profile record
Resources		
POST /api/resources	No	Create a new resource entry
GET /api/resources/schema	No	Get resource filter schema definition
GET /api/resources/{id}	No	Get resource by id with facets
GET /api/resources/search	No	Search resources with filters and paging
GET /api/resources/lookup	No	Lookup resources by domain list
GET /api/resources/{id}/rating	No	Get aggregate rating for resource
GET /api/resources/{id}/threads	No	List discussion threads for resource
POST /api/resources/{id}/threads	Yes	Create discussion thread for resource
POST /api/threads/{id}/reply	Yes	Reply to a discussion thread
GET /api/resources/{id}/save-state	Yes	Get current user's save state
POST /api/resources/{id}/save	Yes	Save resource for current user
DELETE /api/resources/{id}/save	Yes	Unsave resource for current user
POST /api/resources/{resourceId}/images	Yes	Upload an image for resource
GET /api/resources/{resourceId}/images	No	List images for a resource
DELETE /api/resources/images/{imageId}	Yes	Delete a resource image record
Reviews		
GET /api/resources/{id}/reviews	No	List reviews for a resource
POST /api/resources/{id}/reviews	Yes	Create a review for resource

Pokračování na další straně

Tabulka 3: Přehled API endpointů aplikace Resourcia

Endpoint	Auth	Popis
PATCH /api/resources/{id}/reviews/{reviewId}	Yes	Update a resource review by id
DELETE /api/resources/{id}/reviews/{reviewId}	Yes	Delete a resource review by id
POST /api/resources/{id}/reviews/{reviewId}/votes	Yes	Vote review helpful or not
DELETE /api/resources/{id}/reviews/{reviewId}/votes	Yes	Remove your vote on review
GET /api/resources/{resourceId}/reviews/current	Yes	Get current user's review for resource
Reviews Legacy		
POST /api/resource/reviews/{reviewId}/vote	Yes	Legacy vote review helpfulness toggle
DELETE /api/resource/reviews/{reviewId}/vote	Yes	Legacy remove review vote toggle
DELETE /api/resource/{id}/reviews	Yes	Legacy delete own resource review
Search		
GET /api/Search/Schema	No	Get searchable filters with data
GET /api/Search/SchemaDebug	No	Debug filter schema statistics summary

3.8 Nasazení aplikace

Nasazení aplikace nebylo v době psaní této práce plně realizováno, avšak byl navržen postup a provedeny kroky, které k tomu vedou.

Pro provoz aplikace byla zakoupena doména `resourcia.net` prostřednictvím služby Squarespace (Squarespace, 2024). Současně byl zakoupen server. Konkrétně bylo nutné zvolit VPS server, nikoliv běžný webhosting, jelikož VPS umožňuje spouštění Docker kontejnerů. Aplikace Resourcia je navržena tak, aby běžela v Docker prostředí, aktuálně v rámci jednoho kontejneru.

Pro zajištění dostupnosti a bezpečnosti systému by bylo nutné implementovat několik opatření:

- **Firewall**: omezení přístupu pouze na porty související s provozem aplikace (např. HTTP/HTTPS) a SSH. Tím je server chráněn před neoprávněnými přístupy.
- **Zálohy**: pravidelné zálohování databáze PostgreSQL, které umožní obnovu dat v případě výpadku nebo jejich ztráty.
- **Monitoring aplikace**: sledování stavu serveru a využití zdrojů (CPU, paměť), což umožňuje včas odhalit problémy nebo podezřelou aktivitu.
- **Error logging**: zaznamenávání a analýza chyb (např. pomocí služby Sentry (Sentry, 2024)).
- **Performance tracking**: sledování rychlosti odpovědí jednotlivých API endpointů za účelem optimalizace výkonu aplikace.
- **Logování requestů**: ukládání informací o příchozích požadavcích (např. endpoint, čas zpracování, status kód), což umožňuje zpětnou analýzu provozu a detekci podezřelých aktivit (např. DDoS útoky nebo pokusy o neoprávněný přístup).

Aplikace by byla veřejně dostupná prostřednictvím internetové domény a přístupná pomocí webového prohlížeče.

3.9 Budoucí rozvoj

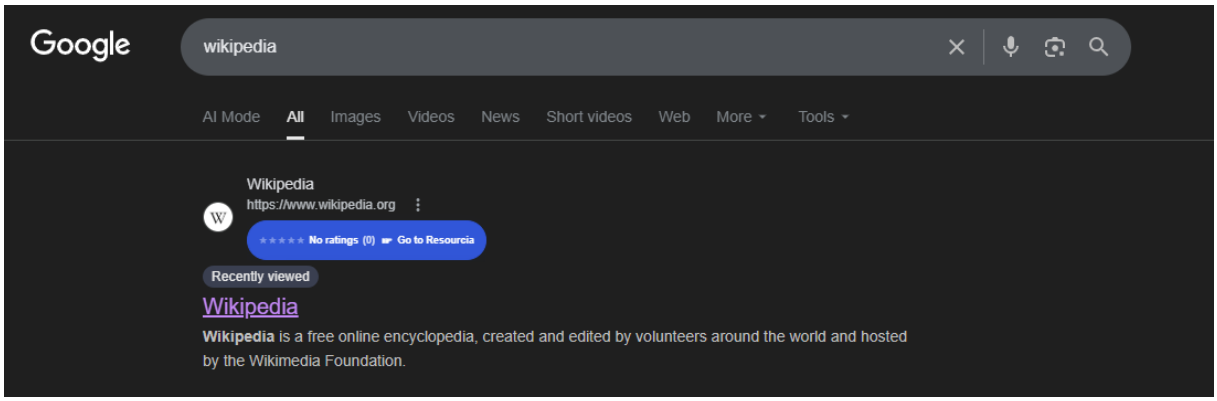
Aplikace Resourcia má značný potenciál dalšího rozvoje. V rámci budoucího vývoje je plánováno několik změn a nových funkcionalit.

Jedním z hlavních směrů je marketing na sociálních sítích. Resourcia je závislá na aktivitě komunity, a proto bude vyžadovat přísun nových uživatelů a administrátorů. Cílem je oslovit studenty prostřednictvím cílených marketingových kampaní. V současné době má Resourcia profil na sociální síti Instagram (`resourcia.official`), který sleduje převážně okruh známých zakladatele.

Z technického hlediska je plánováno dokončení funkcionality nahrávání obrázků. Tato funkce by umožnila lepší rozlišení jednotlivých zdrojů a zároveň by uživatelům umožnila nastavit si profilové fotografie.

Dalším krokem je plné zprovoznění rozšíření pro prohlížeč Google Chrome, které aktuálně slouží pouze k zobrazení hodnocení daného odkazu.

Dále je plánována spolupráce se studentskou komunitou STEM_HUB, která sdružuje studenty se zájmem o vědu a technologie. Tato spolupráce by byla založena na vzájemné



Obrázek 7: Příkladná ukázka rozšíření Chrome

podpoře – STEM_HUB sdružuje motivované studenty, kteří hledají kvalitní zdroje pro samostudium, zatímco Resourcia poskytuje nástroj pro jejich vyhledávání a organizaci. Spolupráce je realistická, jelikož mám v komunitě osobní kontakty na organizátory a obdržel jsem pozitivní zpětnou vazbu od zakladatele Johnathana Skrly. Tato příležitost by mohla vést k realizaci prvního beta testování aplikace v rámci komunity STEM_HUB. Ukázka e-mailové komunikace je uvedena v příloze.

3.9.1 Ukázka e-mailové komunikace

E-mail odeslaný komunitě STEM_HUB

> Ahoj, STEM_HUB!
>
> Doufám, že máte skvělý víkend. Jsem Matěj Hladeček ze 1. ITG. Již několik měsíců se účastním vašich setkání a aktivně přispívám na Discordu, takže komunitu znám a sleduji její dění.
>
> Při sledování diskuzí jsem si všiml, že spousta skvělých příležitostí a zdrojů se po čase ztrácí v historii chatu. To mě vedlo k založení Resourcie. Je to vyhledávač samostudijních zdrojů na internetu pro středoškolské studenty. Cílem není nahrazovat komunity jako STEM_HUB, ale usnadnit studentům orientaci a opakovaný přístup k užitečným materiálům.
>
> Projekt se blíží k první produkční verzi. Uživatelé budou moci vyhledávat zdroje podle tématu a zájmů a postupně do něj přidávám další obsah, aby bylo objevování nových materiálů co nejjednodušší. Pro představu přikládám několik screenshotů.
>
> Rád bych probral možné způsoby spolupráce se STEM_HUB – například sdílení vybraných zdrojů, testování aplikace členy komunity nebo společné projekty. Pokud máte zájem, můžeme se domluvit na krátkém callu nebo testu

aplikace s pár členy vaší komunity. Také budu rád za jakékoliv nápady či doporučení, jak projekt dál rozvíjet.

Děkuji za váš čas a těším se na zpětnou vazbu.

S pozdravem,
Matěj Hladeček
Resourcia

Odpověď

Ahoj!

Tvůj nápad je skvělý, a věřím že určitě vymyslíme spolupráci, která by dávala smysl pro obě dvě strany. Jakmile mi to trochu více přiblížíš, tak věřím, že najdu i další organizace, které by o to mohly mít zájem.

Navrhneš nějaké termíny, které se ti hodí? Já mohu víceméně vždy v podvečer od 19:00, až na středu a čtvrtek (to bych mohl až po deváté), nebo případně ještě dnes:)

Přeji hezký zbytek dne!

4 Závěr

Cílem této práce bylo navrhnout, naprogramovat a implementovat webovou aplikaci Resourcia. Jedná se o platformu, která umožňuje filtrované vyhledávání, sdílení a hodnocení vzdělávacích zdrojů na internetu. Tento cíl byl splněn.

V teoretické části byl analyzován současný stav vzdělávání a dopady internetu na něj. Bylo prokázáno několik pozitivních vztahů a souvislostí, které poukazují na užitečnost online studia. Na druhou stranu bylo odhaleno několik nedostatků, které souvisejí s fenoménem tzv. tutorial hell. Tyto problémy motivovaly vznik projektu Resourcia. Na základě analýzy stávajících řešení, jako jsou YouTube, Stack Overflow nebo Khan Academy, byly definovány požadavky očekávané od navrhovaného řešení.

Praktická část se zabývala implementací aplikace jako takové. Byl navržen databázový model, který pokrývá všechny hlavní entity očekávané v systému. Pro tento projekt byl zvolen technologický stack tvořený backendem v jazyce C# a frontendem vytvořeným ve frameworku Angular, stylovaným pomocí Tailwind CSS. Systém využívá cache vrstvu Redis pro optimalizaci rychlosti odpovědí přicházejících z databáze. V rámci systému byly integrovány i externí služby, jako jsou Cloudflare CAPTCHA, Google OAuth a MailTrap. Aplikace je navíc kontejnerizována pomocí Dockeru, což umožňuje její nasazení na VPS s

veřejně dostupnou doménou (resourcia.net).

Výsledkem práce je funkční webová aplikace splňující stanovené požadavky. Navíc v úvodních pokusech o propagaci a spolupráci byl nápad přijat převážně s nadšením. Práce na projektu přinesla cenné zkušenosti v oblasti full-stack vývoje a návrhu systémové architektury.

Do budoucna je plánováno implementovat funkce, jako je nahrávání obrázků přes frontend, plné zprovoznění Chrome rozšíření a první pokus o beta testování. Aplikace tak může v blízké budoucnosti dále naplňovat svůj účel – předávání hodnotných zdrojů mezi uživateli.

5 Zdroje

Odkazy

[1] ACAR, Yasemin et al. You get where you're looking for: The impact of information sources on code security. *IEEE Symposium on Security and Privacy*, 2017.

[2] ANGULAR. *Angular Documentation* [online]. 2024. Dostupné z: `https://angular.io/docs` [cit. 2026-04-01].

[3] ANGULAR. *Angular Documentation* [online]. 2024. Dostupné z: `https://angular.dev/` [cit. 2026-04-01].

[4] BEM. *BEM Methodology* [online]. 2024. Dostupné z: `https://getbem.com/` [cit. 2026-04-01].

[5] BOOTSTRAP. *Bootstrap Documentation* [online]. 2024. Dostupné z: `https://getbootstrap.com/docs/` [cit. 2026-04-01].

[6] CLOUDFLARE. *Turnstile CAPTCHA* [online]. 2024. Dostupné z: `https://developers.cloudflare.com/turnstile/` [cit. 2026-04-01].

[7] DOCKER. *Docker Documentation* [online]. 2024. Dostupné z: `https://docs.docker.com/` [cit. 2026-04-01].

[8] FIELDING, Roy T. *Architectural Styles and the Design of Network-based Software Architectures*. Disertační práce. 2000.

[9] FONT AWESOME. *Font Awesome Icons* [online]. 2024. Dostupné z: `https://fontawesome.com/` [cit. 2026-04-01].

[10] FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[11] FOWLER, Martin. *Microservices and Modular Architecture* [online]. 2015. Dostupné z: `https://martinfowler.com/` [cit. 2026-04-01].

[12] GAMMA, Erich et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[13] GIT. *Git Documentation* [online]. 2024. Dostupné z: `https://git-scm.com/docs` [cit. 2026-04-01].

[14] GITHUB. *About GitHub* [online]. 2024. Dostupné z: `https://github.com/about` [cit. 2026-04-01].

[15] GOOGLE. *OAuth 2.0* [online]. 2024. Dostupné z: https://developers.google.com/identity/protocols/oauth2 [cit. 2026-04-01].

[16] GOOGLE. *Google Fonts* [online]. 2024. Dostupné z: https://fonts.google.com/ [cit. 2026-04-01].

[17] INDEED HIRING LAB. *The Declining Demand for Bachelor's Degrees in Job Postings* [online]. 2024. Dostupné z: https://www.hiringlab.org/2024/02/27/educational-requirements-job-postings/ [cit. 2026-04-01].

[18] JONES, Michael et al. *JSON Web Token (JWT)*. RFC 7519, 2015.

[19] JONES, Michael et al. *JSON Web Token (JWT)*. RFC 7519, 2015.

[20] KHAN ACADEMY. *About Khan Academy* [online]. 2024. Dostupné z: https://www.khanacademy.org/about [cit. 2026-04-01].

[21] KIRSCHNER, Paul A., SWELLER, John a CLARK, Richard E. Why minimal guidance during instruction does not work. *Educational Psychologist*, 2006.

[22] MAILTRAP. *Mailtrap Email Testing* [onli]() . 2024. Dostupné z: https://mailtrap.io/ [cit. 2026-04-01].

[23] MCKINSEY GLOBAL INSTITUTE. *The Great Transformer* [online]. 2011. Dostupné z: https://www.mckinsey.com/industries/technology-media-and-telecommunications/our- [cit. 2026-04-01].

[24] MICROSOFT. *.NET Documentation* [online]. 2024. Dostupné z: https://learn.microsoft.com/en-us/dotnet/ [cit. 2026-04-01].

[25] MITRA, Sugata. Self-organising systems for mass computer literacy. *International Journal of Development Issues*, 2005.

[26] NG-EXTRACT-I18N-MERGE. *ng-extract-i18n-merge* [online]. 2024. Dostupné z: https://www.npmjs.com/package/ng-extract-i18n-merge [cit. 2026-04-01].

[27] OECD. *PISA 2022 Results* [online]. 2022. Dostupné z: https://www.oecd.org/pisa/ [cit. 2026-04-01].

[28] OWASP. *Web Security Testing Guide* [online]. 2023. Dostupné z: https://owasp.org/ [cit. 2026-04-01].

[29] PEW RESEARCH CENTER. *The Internet and Social Mobility* [online]. 2021. Dostupné z: https://www.pewresearch.org/internet/ [cit. 2026-04-01].

[30] POSTGRESQL. *PostgreSQL Documentation* [online]. 2024. Dostupné z: https://www.postgresql.org/docs/ [cit. 2026-04-01].

[31] REDDIT. *About Reddit* [online]. 2024. Dostupné z: https://www.redditinc.com/ [cit. 2026-04-01].

[32] REDIS. *Redis Documentation* [online]. 2024. Dostupné z: https://redis.io/docs/ [cit. 2026-04-01].

[33] RESCORLA, Eric. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, 2018.

[34] ROBINS, Anthony, ROUNTREE, Janet a ROUNTREE, Nathan. Learning and teaching programming: A review and discussion. *Computer Science Education*, 2003.

[35] SASS. *Sass Documentation* [online]. 2024. Dostupné z: https://sass-lang.com/documentation/ [cit. 2026-04-01].

[36] SAVE MY EXAMS. *Online Learning vs Classroom Statistics* [online]. 2024. Dostupné z: https://www.savemyexams.com/learning-hub/insights/online-learning-vs-classroom-sta [cit. 2026-04-01].

[37] SENTRY. *Application Monitoring Platform* [online]. 2024. Dostupné z: https://sentry.io/ [cit. 2026-04-01].

[38] STACK EXCHANGE. *About Stack Exchange* [online]. 2023. Dostupné z: https://stackexchange.com/sites [cit. 2026-04-01].

[39] STACK OVERFLOW. *Developer Survey 2023* [online]. 2023. Dostupné z: https://survey.stackoverflow.co/2023/ [cit. 2026-04-01].

[40] SQUARESPACE. *Domains & Website Hosting* [online]. 2024. Dostupné z: https://www.squarespace.com/ [cit. 2026-04-01].

[41] SZYPERSKI, Clemens. *Component Software: Beyond Object-Oriented Programming*. 2. vydání. Addison-Wesley, 2002.

[42] TAILWIND CSS. *Tailwind CSS Documentation* [online]. 2024. Dostupné z: https://tailwindcss.com/docs [cit. 2026-04-01].

[43] TANENBAUM, Andrew S. a WETHERALL, David J. *Computer Networks*. 5. vydání. Pearson, 2016.

[44] TESTGORILLA. *The State of Skills-Based Hiring 2024* [online]. 2024. Dostupné z: https://www.testgorilla.com/skills-based-hiring/state-of-skills-based-hiring-2024/ [cit. 2026-04-01].

[45] TUNKELANG, Daniel. *Faceted Search*. Morgan & Claypool, 2009.

[46] WORLD BANK. *Information and Communications for Development 2009* [online]. 2009. Dostupné z: `https://documents.worldbank.org/` [cit. 2026-04-01].

[47] WORLD BANK. *Education and Technology* [online]. 2023. Dostupné z: `https://www.worldbank.org/en/topic/edutech` [cit. 2026-04-01].

[48] YOUTUBE. *YouTube for Press* [online]. 2024. Dostupné z: `https://www.youtube.com/intl/en-GB/about/press/` [cit. 2026-04-01].